

0.04%

A Decentralized Carbon Credit Marketplace

Blockchain, Computer Vision, and Decentralized Storage for Transparent Carbon Credit Verification

PROJECT REPORT

Prepared by: Aarush Agarwal

Table of Contents

1. Executive Summary
 2. Problem Statement
 3. Proposed Solution & Objectives
 4. System Architecture
 5. Technology Stack
 6. Smart Contract Layer
 7. Machine Learning & Verification Pipeline
 8. Frontend Application
 9. Decentralized Storage (IPFS)
 10. Limitations and Known Issues
 11. Future Scope
 12. Conclusion
- Appendix A: Repository Structure
- Appendix B: Glossary

1. Executive Summary

0.04% is a decentralized carbon credit marketplace that combines blockchain, satellite-image-based deep learning, and decentralized storage to address transparency and verification failures in the conventional carbon credit market. The name references the fraction of Earth's atmosphere made up of CO₂ — the problem the project is built around.

Instead of relying on centralized registries and paid third-party verifiers, the system issues an ERC-20 token ("Carbon Coin" for organizations, "EcoCoin" for individuals) only after a reforestation claim clears an automated or community-driven verification step. Verification is anchored in a YOLOv8 object-detection model that counts trees in before/after satellite imagery, feeding a formula that estimates the carbon impact of a claim. Claim evidence is pinned to IPFS rather than stored centrally, and the resulting credits are tradable through an on-chain marketplace.

This report documents the problem being solved, the architecture that was built, the specific mechanics of each component, and an honest account of what remains prototype-grade versus what would need further work before real-world deployment.

2. Problem Statement

The voluntary carbon credit market is the primary financial mechanism for funding reforestation and other offset projects, but its centralized structure creates four recurring failure modes that directly motivated this project:

2.1 Lack of Transparency

Credit issuance decisions are typically made by a small number of private registries whose verification methodology and underlying evidence are not publicly auditable. Buyers of credits have limited ability to independently confirm that a claimed offset actually occurred.

2.2 High Intermediation Costs

Verification bodies, brokers, and registries each take a cut of the transaction, which disproportionately burdens small-scale reforestation developers — often the actors closest to the land and the ones the market is meant to incentivize.

2.3 Price Manipulation and Inconsistent Regulation

Carbon credit prices vary significantly across registries and jurisdictions, and are vulnerable to speculative trading disconnected from the underlying environmental impact, creating pricing uncertainty for genuine buyers.

2.4 Slow Verification and Fraud Risk

Manual, document-based verification is slow, and the absence of a shared, tamper-evident ledger makes double-counting of the same offset across multiple registries difficult to prevent.

3. Proposed Solution & Objectives

The project's core thesis is that verification and issuance can be decentralized without sacrificing rigor, by combining three mechanisms:

- On-chain issuance: an ERC-20 token is minted only after a claim passes verification, and every mint, claim, and trade is recorded on an immutable, publicly queryable ledger.
- Automated, imagery-based verification: rather than trusting a claimant's self-reported numbers, before/after satellite imagery is passed through an object-detection model to independently estimate change in tree cover.
- Tamper-evident evidence storage: claim photos and documentation are pinned to IPFS, a content-addressed storage network, so evidence cannot be silently swapped or edited after submission.

Design objectives set out for the system:

- Remove reliance on a single centralized verifier by giving the option of either automated (imagery-based) or community (peer-voted) verification.
- Make every claim, vote, and token mint publicly auditable on-chain.
- Keep the barrier to entry low enough for small-scale developers by avoiding broker fees.
- Provide a liquid, standardized market for the resulting credits rather than opaque bilateral deals.

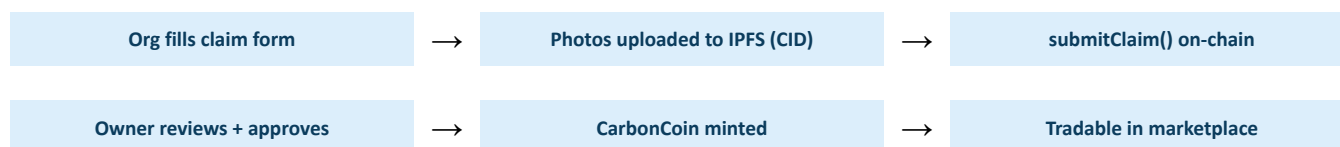
4. System Architecture

The system is organized into four cooperating layers, summarized below and then detailed in the sections that follow.

Layer	Responsibility	Key Technologies
Frontend	Wallet connection, claim submission forms, marketplace browsing, org/user registration	React 18, Vite, React Router, Tailwind CSS, wagmi, RainbowKit/ConnectKit, ethers.js
Smart Contracts	Token issuance, claim lifecycle (submit → verify → mint), credit trading, on-chain reputation	Solidity ^0.8.19, OpenZeppelin (ERC-20, Ownable, ReentrancyGuard)
ML / Verification	Detect tree count from satellite imagery; estimate vegetation-cover change between two dates	YOLOv8 (Ultralytics), OpenCV / scikit-image, Roboflow-hosted dataset
Backend / Storage	Serve model predictions over HTTP; pin and retrieve claim evidence via IPFS	Flask + Flask-CORS (inference API); Node.js/Express + Multer + Lighthouse SDK (IPFS gateway)

4.1 End-to-End Claim Flow (Organization Path)

The diagram below traces a single organization claim from submission to tradable token, the primary path implemented in this codebase:



In detail: the organization completes a claim form in the React frontend (GPS coordinates, acreage, requested token amount, project narrative) → supporting photos are uploaded through the Express/Lighthouse microservice, which returns a content hash (CID) → the claim, including the IPFS CIDs, is submitted to CarbonMarketplace.sol → the contract owner (acting as verifier/admin), optionally cross-referencing the tree-detection model's output from the Flask inference API, approves the claim for an eligible token amount → CarbonCoin (ERC-20) is minted directly to the organization's wallet and becomes sellable to other registered organizations through the on-chain marketplace.

4.2 End-to-End Claim Flow (Peer-Voting Path)

A second, parallel path targets individual users and replaces admin approval with a time-boxed community vote:



5. Technology Stack

Category	Tools / Libraries	Purpose in this project
Smart contracts	Solidity ^0.8.19, OpenZeppelin (ERC20, Ownable, ReentrancyGuard)	Token issuance, claim lifecycle, marketplace logic
Frontend framework	React 18, Vite, React Router DOM v7	SPA structure, routing between org/user flows
Styling	Tailwind CSS 3, custom CSS (Navbar)	Utility-first responsive styling
Web3 connectivity	ethers.js v6, wagmi, RainbowKit, ConnectKit, viem	Wallet connection, contract calls, transaction signing
Machine learning	YOLOv8m (Ultralytics), OpenCV, scikit-image	Tree detection / counting from satellite imagery
Dataset tooling	Roboflow (hosted annotation + export)	Source and format of the single-class "tree" dataset
Inference backend	Flask, Flask-CORS	HTTP endpoint exposing the trained model as a prediction service
Decentralized storage	IPFS, Lighthouse Web3 SDK, Node.js/Express, Multer	Pinning and retrieving claim evidence off-chain

6. Smart Contract Layer

Two independent contract systems were implemented, reflecting two different trust models for verification: administrator-approved organizational claims, and peer-voted individual claims. Both mint an ERC-20 token only on approval, and both build on OpenZeppelin's audited ERC-20, Ownable, and ReentrancyGuard primitives rather than custom security-critical code.

6.1 CarbonCoin / CarbonMarketplace (contracts/CarbonCoinsMarketplace.sol)

Targets organizations. CarbonToken is a standard ERC-20 ("CarbonCoin", ticker CC) mintable only by the marketplace contract. Organizations register once with a name, self-declared net emissions figure, and an IPFS-hosted profile photo, and begin with a starting wallet balance for the demo economy.

Key state

- Organization { name, netEmission, photoIpfsHash, isRegistered, balance, wallet } — balance is the org's CarbonCoin holding; wallet is a separate mock currency balance used to purchase credits from other organizations.
- Claim { id, seller, coordinatesX, coordinatesY, acres, demandedTokens, eligibleTokens, projectDetails, projectName, timestamp, photoIpfsHashes[], startYear, status } — status is Pending, Approved, or Declined.
- MOCK_CARBOIN_PRICE — a fixed reference price (50 CC per credit) used by the trading function.

Key functions

Function	Access	Effect
registerOrganization(name, netEmission, photoHash)	Public	One-time registration; reverts if the caller is already registered.
submitClaim(x, y, acres, demandedTokens, details, name, photoHashes[], startYear)	Registered orgs	Creates a Pending claim; emits ClaimSubmitted.

Function	Access	Effect
approveClaim(claimId, eligibleTokens)	Owner only	Mints eligibleTokens ($\leq$ demandedTokens) of CarbonCoin to the claimant and marks the claim Approved.
declineClaim(claimId)	Owner only	Marks the claim Declined; eligibleTokens set to 0.
purchaseCredits(seller, amount)	Registered orgs	Transfers CarbonCoin from seller to buyer at MOCK_CARBON_PRICE per unit; guarded with nonReentrant.
getOrganization / getClaim / getBalance / getCarbonPrice	View	Read-only accessors used by the frontend.

6.2 EcoCoin / EcoCoinMarketplace (contracts/EcoCoin.sol)

Targets individual users and replaces administrator approval with a time-boxed community vote, aiming to make verification permissionless rather than gatekept by a single owner address.

Key state and constants

- VOTING_DURATION = 5 minutes (shortened for demo purposes; would be days/weeks in production).
- REWARD_AMOUNT = 2 EcoCoin, paid to voters on the winning side of a resolved claim.
- COINS_PER_CREDIT = 100,000 — the conversion rate between EcoCoin units and one "carbon credit" equivalent.
- User { name, photoHash, totalClaimsSubmitted, successfulClaims, totalVotesCast, correctVotes, isRegistered, reputationScore } — reputation starts at 100, +5 per correct outcome (capped at 1000), -3 per incorrect outcome (floored at 0).

Key functions

Function	Access	Effect
registerUser(name, photoHash)	Public	One-time registration with a starting reputation of 100.
submitClaim(name, photoHash, claimType, expectedCoins)	Registered users	Opens a 5-minute voting window; rate-limited to one claim per user per day.
vote(claimId, voteFor)	Registered users (not claim owner)	Casts a for/against vote once, only during the voting window.
resolveClaim(claimId)	Anyone, after voting ends	Determines majority outcome; mints expectedCoins to the claimant if approved, then calls distributeRewards.
distributeRewards(claimId)	Internal	Rewards voters who sided with the majority outcome with 2 EcoCoin and a reputation increase; see Section 11 for an important caveat on the current implementation.
updateReputation(user, positive)	Internal	Applies the $\pm 5/-3$ reputation adjustment described above.

Both contracts follow the checks-effects-interactions pattern for state updates and use OpenZeppelin's ReentrancyGuard on functions that move tokens between parties (purchaseCredits), which is the standard mitigation for reentrancy attacks in token-transfer functions.

7. Machine Learning & Verification Pipeline

Verification of environmental impact is anchored in a computer-vision pipeline rather than manual inspection or self-reported figures.

7.1 Detection Model

- Architecture: YOLOv8m (Ultralytics), a single-stage, medium-sized object detector, fine-tuned from COCO-pretrained weights (yolov8m.pt).
- Task: single-class detection ("tree") — the model outputs a bounding box per detected tree in a given satellite/aerial image.
- Dataset: sourced and annotated via Roboflow (project tree-counting-qi3h). The local export used for training contains 19 training images and 9 validation images — a small, hackathon-scale sample rather than a production dataset.
- Training configuration: 10 epochs, batch size 16, image size 640×640, automatic optimizer selection, default augmentation (mosaic, closed over the final 10 epochs).

7.2 Training Results

Metrics from the final training run (train14):

Metric	Value (final epoch)	Interpretation
Precision	0.595	~60% of predicted tree boxes were correct at the default confidence threshold.
Recall	0.544	~54% of actual trees in validation images were detected.
mAP@0.5	0.518	Moderate detection quality at a relaxed IoU threshold.
mAP@0.5:0.95	0.203	Substantially lower under stricter IoU averaging — indicates imprecise box localization, expected with this little training data.

The Ultralytics run directory also includes standard diagnostic artifacts — precision/recall curves, a confusion matrix, and labeled training-batch mosaics — consistent with a normal (if brief) YOLO fine-tuning run. Multiple earlier training runs (train2 through train13) are present in the repository, indicating iterative hyperparameter/data experimentation before settling on the reported configuration.

7.3 Carbon Credit Estimation Formula

The inference backend (backend/app.py) converts a pair of before/after images into an estimated Carbon Coin allocation:

- Both images are run through the trained model at a low confidence threshold (0.1) to maximize recall of small/occluded trees.
- Each detected tree is assumed to occupy a fixed canopy area of 9.3 m², converting a tree count into an estimated vegetated area.
- The percentage change in tree count between the two dates is computed as $(\text{trees_after} - \text{trees_before}) / \text{trees_before} \times 100$.
- This percentage change, the claimed acreage, the number of years elapsed since the project's declared start year, and an assumed average NDVI of 0.6, are combined into a single scalar output representing the estimated Carbon Coin value of the claim.

```
cc = area * 4.0486 * (1 + pct_change/200) * (2025 - start_year) * (1 + avg_ndvi) / 100
```

This is exposed over HTTP via a single Flask route:

```
POST /predict { latitude, longitude, area, year } → { status, prediction }
```

This formula is a deliberately simple, hackathon-stage heuristic rather than a scientifically calibrated carbon-sequestration model — the fixed canopy-area and NDVI assumptions are flagged explicitly in Section 11.

8. Frontend Application

The frontend is a Vite + React single-page application. Routing and page-level structure are defined in App.jsx, wrapped in a global Web3Provider so wallet state is available to every route.

8.1 Routes and Pages

Route	Component	Purpose
/	LandingPage (Hero + About sections)	Marketing landing page with a looping background video, headline, and mission statement.
/organisation/register	OrganisationRegisterPage	Org onboarding: name, net emissions, IPFS-uploaded profile photo, on-chain registerOrganization() call.
/organisation/marketplace	OrganisationMarketplace	Lists registered organizations (queried via the OrganizationRegistered event log) with photo, address, and CarbonCoin balance.
/organisation/claim	OrganisationClaimPage	Claim submission form: coordinates, acreage, demanded tokens, project details, IPFS photo upload, on-chain submitClaim() call.
/register	RegisterPage	Entry point for choosing between organization and individual (EcoCoin) registration flows.

Individual-user routes for the EcoCoin path (UserRegisterPage, UserClaimSubmissionPage under a separate UserWeb3Provider) exist in the codebase but are commented out of the active router in App.jsx — the organization path is the one wired up end-to-end in the current build.

8.2 State & Wallet Management

- Web3Context (hooks/Web3hook.jsx) wraps ethers.js's BrowserProvider around window.ethereum (MetaMask), exposing walletAddress, signer, contract instance, and the caller's registered organization record to every component.
- IpfsContext (context/IpfsContext.jsx) centralizes file upload/retrieval calls to the local Express/Lighthouse microservice, with shared loading and error state, plus basic client-side file-size (10 MB) and file-type validation before upload.
- wagmi, RainbowKit, and ConnectKit are included as dependencies for broader wallet support beyond raw MetaMask, though the primary connection path exercised in the current pages is the direct ethers.js/MetaMask flow.

8.3 Styling

Tailwind CSS is used throughout for layout and responsive design, with a dark theme (gray-900/black backgrounds, yellow/orange accent color) consistent with the "0.04% CO₂" branding. The landing page's hero section uses a looped background video and large gradient-clipped typography; the Navbar conditionally hides itself on registration routes and highlights the active route.

9. Decentralized Storage (IPFS)

Claim evidence — photos and project documentation — is stored off-chain on IPFS rather than on the blockchain itself, keeping on-chain storage costs low while preserving content-addressed, tamper-evident references: a file's IPFS hash (CID) changes if the file's contents change, so a CID recorded in a claim can be used to verify the evidence hasn't been swapped after the fact.

9.1 Upload/Retrieval Service

A lightweight Express server (ipfs/server.mjs) sits between the frontend and IPFS:

Endpoint	Method	Behavior
/send	POST (multipart, field "file")	Accepts a file via Multer (in-memory, 10 MB limit), pins it to IPFS via the Lighthouse SDK, and returns the resulting CID.
/get/:cid	GET	Resolves a CID through the Lighthouse gateway and redirects/returns the underlying file.

The frontend's IPFS context wraps these two endpoints, so components never talk to Lighthouse directly — file upload from the claim/registration forms and file retrieval in the marketplace listing both go through this single service, which also centralizes CORS handling and upload-progress reporting.

10. Limitations and Known Issues

As a hackathon-stage prototype, several parts of the system are intentionally simplified or incomplete. Stating these plainly is more useful for an internship report than glossing over them:

- Voter-reward loop is a placeholder: `distributeRewards()` in `EcoCoin.sol` iterates over an address range cast from claim timestamps rather than an actual list of voters — this is explicitly marked as a placeholder in the source and would need a proper voter registry (e.g., an array or mapping populated in `vote()`) before the reward/reputation mechanism is functionally correct, since the current loop will not reliably reach real voter addresses and would be prohibitively gas-expensive if it did.
- Carbon-estimation formula relies on fixed assumptions: a flat 9.3 m² canopy area per detected tree and a constant average NDVI of 0.6, rather than deriving either from the imagery itself. A production version would compute NDVI directly from multispectral satellite bands and calibrate canopy area against ground-truth measurements.
- Small training dataset: the YOLOv8 model was trained on only 19 training / 9 validation images for 10 epochs, giving moderate precision/recall (Section 7.2) — sufficient to demonstrate the pipeline end to end but not validated at production scale.
- Fixed marketplace pricing: the credit trading price (50 CC per credit) is a hardcoded mock value rather than a market-driven or auction-based price.
- Local file-path dependencies: the inference script currently reads two static, locally-pathed sample images (`Sample1.png` / `Sample2.png`) rather than accepting arbitrary claim imagery dynamically through the `/predict` endpoint's request payload.
- Single point of trust in the organization path: claim approval in `CarbonMarketplace.sol` is gated entirely behind `onlyOwner`, meaning the "decentralization" of verification in that path is currently procedural (an admin manually reviews model output) rather than automated or multi-party.
- `EcoCoin`'s individual-user flow is implemented but not wired into the live router (Section 8.1), so it has not been exercised end-to-end in the same way as the organization flow.

11. Future Scope

Extensions identified during the project but not implemented within the hackathon timeline:

- Replace the placeholder voter-reward loop with a proper on-chain voter registry.
- Derive NDVI directly from multispectral imagery rather than using a fixed assumption, and calibrate canopy-area-per-tree against ground-truth field data.
- Expand the training dataset substantially and validate detection accuracy against held-out, geographically diverse imagery.
- Replace fixed marketplace pricing with a genuine on-chain order book or automated market maker (AMM) for price discovery.
- NLP-based verification of submitted project documentation, layered on top of the existing image-based verification.
- Extend beyond reforestation to other offset project types, such as solar and hydroelectric projects, as noted in the original project README.
- Move claim approval in the organization path from a single onlyOwner admin to a multi-signature or DAO-style governance structure, to remove the remaining centralized trust point.

12. Conclusion

0.04% demonstrates a working, end-to-end path from satellite-image-based environmental verification to on-chain token issuance and trading, integrating computer vision, smart-contract engineering, and decentralized storage into a single application within a hackathon timeframe. Two parallel issuance models — administrator-reviewed organizational claims and peer-voted individual claims — illustrate two different trust structures for the same underlying problem. While several components remain at prototype fidelity (Section 11), the project provides a coherent, technically grounded architecture for reducing reliance on manual, broker-mediated carbon credit verification, and a concrete base for further development toward a production-grade decentralized carbon market.

Appendix A: Repository Structure (Abridged)

```

0.04-percent/
├── src/
│   ├── pages/CarbonCredit/      React frontend
│   ├── pages/EcoCoin/          Org register / marketplace / claim pages
│   ├── pages/Home/             Individual user register / claim pages
│   ├── hooks/                  Landing page (hero + about sections)
│   ├── context/                Web3hook.jsx, Web3UserHook.jsx
│   ├── utils/                  IpfsContext.jsx
│   └── constants.jsx (ABIs), ifpsUpload.jsx
├── contracts/
│   ├── CarbonCoinsMarketplace.sol  Org-facing marketplace + CarbonToken
│   └── EcoCoin.sol                 Peer-voting marketplace + EcoCoin
├── backend/
│   ├── app.py                  Flask inference API (/predict)
│   └── requirements.txt
├── tree_detection/
│   ├── Dataset/                Roboflow-exported train/valid images + labels
│   └── PythonScripts/runs/      YOLOv8 training runs and weights
├── ipfs/
│   └── server.mjs               Express + Lighthouse SDK upload/retrieval API
└── README.md

```

Appendix B: Glossary

Term	Meaning in this project
ERC-20	The standard Ethereum token interface used for both CarbonCoin and EcoCoin.
NDVI	Normalized Difference Vegetation Index — a satellite-imagery metric of vegetation health/density; used here as a fixed assumption (0.6) rather than computed.
IPFS / CID	InterPlanetary File System, a content-addressed storage network; a CID is the content hash used to reference an uploaded file.
mAP@0.5 / mAP@0.5:0.95	Mean Average Precision — standard object-detection accuracy metrics, evaluated at a single IoU threshold (0.5) or averaged across a range (0.5–0.95).
YOLOv8	"You Only Look Once" v8 — a single-stage real-time object-detection model family (Ultralytics), used here to detect trees.
Claim	A submission (by an organization or individual) requesting token issuance in exchange for evidence of a carbon-offsetting activity.
Reentrancy Guard	A smart-contract security pattern (OpenZeppelin's ReentrancyGuard) preventing a function from being re-entered mid-execution — used on token-transfer functions.